SERVICENOW INCIDENT MANAGEMENT

Implement Client Script & UI Policy (Incident)

Platform: ServiceNow

Module: Incident Management

Project Type: Micro Project

Core Topics: UI Policy, UI Policy Actions, Client Scripts, Form Validation

1. Executive Summary

This report provides a detailed executive-level summary of the ServiceNow Incident Management micro project titled “Implement Client Script & UI Policy (Incident).” The implementation demonstrates how client-side configuration can be used to enforce consistent Incident data entry, automate selected field values, control field behavior according to Incident Impact, validate information before a record is submitted, and restrict direct State changes made through list editing.

The solution combines one UI Policy, two UI Policy Actions, and three Client Script types: onChange, onSubmit, and onCellEdit. Together, these controls establish a defined workflow for High-impact Incidents and provide specific safeguards against incomplete or inappropriate updates.

The central business purpose is data integrity. Incident records are used for triage, routing, resolution, reporting, and SLA-related processes; therefore, inconsistent or incomplete information can affect downstream activities. The project addresses this risk at the user-interface level by applying conditional mandatory behavior, read-only behavior, automated values, save-time validation, and controlled State updates.

The documented configuration is intentionally lightweight. It focuses on the specific rules defined in the project specification rather than attempting to implement a complete enterprise-wide Incident governance framework. The report therefore distinguishes clearly between the controls demonstrated by the project and additional validation or governance activities that may be appropriate before broader deployment.

2. Situation and Business Context

Incident Management depends on users entering complete and consistent information. The supplied project identifies a specific problem: relying only on user awareness and manual checking can result in incomplete, inconsistent, or incorrect Incident information. Such weaknesses can affect triage, routing, resolution, reporting, and SLA-related processes.

The project responds to this situation by moving selected controls into the Incident user interface. Instead of depending entirely on users to remember every requirement, the form dynamically responds to the Impact value. When Impact is High, additional controls are activated. Assignment Group becomes mandatory, Urgency is made

read-only, and Urgency is automatically set to High. Before submission, the configuration also checks that Assigned To is populated.

A separate list-editing control addresses State. The project prevents users from changing State directly through Incident list editing and instead directs them to open the Incident record. State changes made through the normal Incident form remain permitted.

This approach creates a more structured interaction model: the Incident form communicates required behavior to the user, automatically handles selected values, and blocks specified invalid actions.

3. Key Findings

3.1 High-Impact Incident Controls

The UI Policy named “High Impact Control” is configured for the Incident table and is active. Its condition is Impact is 1 – High. The policy uses Reverse if false, meaning that when the Impact condition is no longer true, the field behavior applied by the policy is reversed.

Under this policy, Assignment group is configured as mandatory. This means that a High-impact Incident requires an Assignment Group value. The Urgency field is configured as read-only while remaining visible. These controls are inherited from the High Impact Control condition.

3.2 Automated Urgency Handling

The onChange Client Script named “Auto set urgency for high impact” runs when the Impact field changes. It ignores loading and empty-value conditions. When the new Impact value is 1, the script sets Urgency to 1 and displays the informational message “Urgency set to High for High impact incident.”

This creates a direct relationship between Impact and Urgency for the High-impact condition. The configuration reduces the need for manual entry and makes the intended urgency value explicit to the user.

3.3 Save-Time Validation

The onSubmit Client Script named “Prevent save if Assigned To missing” provides a final form-level validation check. If Impact is 1 and Assigned To is empty, the script displays an error against Assigned To and returns false. Returning false prevents the Incident from being saved.

When the required Assigned To value is supplied, the validation condition is not triggered and the form can proceed with the save operation. This complements the dynamic UI behavior by providing an explicit submission-time control.

3.4 Controlled State Updates

The onCellEdit Client Script named “Prevent state change via list edit” applies to the State field. If a user attempts to edit State directly from the Incident list, the script displays an alert explaining that State cannot be updated using list editing and instructs the user to open the Incident. The callback is then returned as false, blocking the change.

The project deliberately distinguishes list-based editing from form-based editing. State changes through the normal Incident form remain allowed, and the documented test case expects the change to be saved successfully.

4. Detailed Solution Configuration

4.1 UI Policy – High Impact Control

Configuration Item	Configured Value
Name	High Impact Control
Table	Incident
Active	True
Condition Field	Impact
Operator	is
Condition Value	1 – High
Reverse if false	True

The UI Policy is the primary conditional control in the implementation. Its purpose is to activate the defined field behavior whenever the Incident Impact is High. Reverse if false ensures that the temporary restrictions do not remain active when the Incident is changed to another Impact level.

4.2 UI Policy Action – Assignment Group

Field Configuration	Value
Field Name	Assignment group
Mandatory	True
Condition	Inherited from High Impact Control

When Impact is High, Assignment Group becomes mandatory. This supports routing and ownership by requiring the Incident to have an Assignment Group before it can be treated as complete under the defined High-impact rules.

4.3 UI Policy Action – Urgency

Field Configuration	Value
Field Name	Urgency
Read-only	True
Visible	Leave unchanged

The Urgency field remains visible to the user, but manual modification is restricted while the High Impact Control policy is active. The value is subsequently controlled by the onChange script for the High-impact condition.

4.4 onChange Client Script (Auto Set Urgency)

Configuration Item	Value
Name	Auto set urgency for high impact
Table	Incident

Type	onChange
Field Name	Impact
Active	True

The script evaluates the new Impact value. If the value is 1, it sets the Urgency field to 1 and presents an informational message. The script also exits when the form is loading or when the new Impact value is empty.

Functionally, this means that the user's selection of High Impact becomes the trigger for the automated urgency behavior. The user does not need to manually select the corresponding urgency value.

4.5 onSubmit Client Script – Save Validation

Configuration Item	Value
Name	Prevent save if Assigned To missing
Table	Incident
Type	onSubmit
Active	True

The validation checks two conditions together: Impact must be High and Assigned To must be empty. If both conditions are true, the form displays an error against Assigned To and prevents submission. If either condition is not true, the function returns true and allows the save process to continue.

4.6 onCellEdit Client Script (Prevent State List Editing)

Configuration Item	Value
Name	Prevent state change via list edit
Table	Incident
Type	onCellEdit
Field Name	State

Active	True

This script is designed specifically for list editing. It prevents the direct modification of State from the Incident list and communicates the intended alternative to the user: open the Incident record and make the change through the form.

5. End-to-End Functional Workflow

The implementation follows a clear sequence from Incident entry through validation and update. First, the user opens or creates an Incident. The system evaluates whether Impact is High. If it is High, the High Impact Control is activated. Assignment Group becomes mandatory, Urgency becomes read-only, and the onChange script automatically sets Urgency to High.

When the user submits the Incident, the onSubmit validation checks whether Assigned To is populated. If Assigned To is missing for a High-impact Incident, the save is blocked and an error is shown. If Assigned To is supplied, the Incident can be saved.

For State updates, the workflow differs according to the interface used. Direct list editing is blocked, while State changes through the normal Incident form are permitted.

When Impact is changed away from High, the UI Policy's Reverse if false behavior reverses the policy-driven restrictions. Assignment Group is no longer mandatory because of this policy, and Urgency becomes editable again.

6. Risks and Control Considerations

6.1 Client-Side Enforcement Boundary

The supplied project specifically demonstrates client-side controls. The document does not establish that equivalent server-side controls or other enforcement mechanisms have been implemented. Therefore, the documented behavior should be

understood as the behavior of the configured user-interface interactions rather than as evidence of universal enforcement across every possible integration or update path.

6.2 Configuration Consistency

The project uses multiple controls that address related business behavior. The UI Policy makes fields mandatory or read-only, the onChange script sets Urgency, and the onSubmit script validates Assigned To. These controls should remain logically consistent if the Incident requirements are changed in the future.

6.3 Testing Evidence

The supplied report specifies functional test scenarios and expected outcomes, but it does not provide actual execution evidence, screenshots of successful tests, test dates, tester names, or production validation results. The documented tests should therefore be treated as a defined validation plan rather than as independent evidence that every scenario has already passed.

6.4 Scope of Business Rules

The project covers the specific fields and actions identified in its specification. It does not document broader Incident governance requirements, integration behavior, data imports, API updates, or other potential paths by which Incident records might be modified. Those areas would need separate assessment if the configuration is intended for a broader production control framework.

7. Functional Testing and Expected Results

Test Case	Action	Expected Result
1 – High Impact Behavior	Set Impact to 1 – High	Assignment Group becomes mandatory; Urgency is set to High and becomes read-only.
2 – Prevent Save Without Assigned To	Leave Assigned To empty and submit	Record is not saved and validation message states that Assigned To is mandatory for High-

		impact Incidents.
3 – Successful Save	Populate Assigned To and submit	Incident saves successfully.
4 – Reverse Condition	Change Impact from High to Medium	Assignment Group is no longer mandatory under the policy and Urgency becomes editable.
5 – Block State List Editing	Edit State directly from Incident list	Alert appears and State remains unchanged.
6 – State Form-Based Update	Open Incident, change State, and update	State change is saved successfully.

These six scenarios collectively test the principal behavior described by the project. They cover both activation and reversal of the UI Policy, failed and successful submission paths, and the difference between list-based and form-based State updates.

8. Recommended Next Steps

1. Execute all six functional test scenarios in the intended ServiceNow environment and record the actual result for each test.
2. Capture supporting evidence for each critical test, such as the High-impact field behavior, validation message, blocked State list edit, and successful form-based State update.
3. Review the relationship between the UI Policy and Client Scripts to ensure that future changes to Impact, Urgency, Assignment Group, or Assigned To requirements do not create conflicting behavior.
4. Assess whether the same business rules need enforcement outside the client interface, particularly for other record-update mechanisms.
5. Confirm that the configured behavior is appropriate for the relevant user roles and Incident workflows before production use.

6. Document configuration ownership and a change-management process so that future modifications to the policy or scripts remain traceable.
7. After deployment, monitor Incident records and user feedback for cases in which the intended High-impact or State controls do not behave as expected.

9. Overall Assessment of the Documented Implementation

The supplied project presents a focused and internally structured example of ServiceNow client-side Incident control. Its strongest feature is the way several mechanisms are combined around a single business condition: Impact = High. The UI Policy controls field behavior, the onChange script automates Urgency, and the onSubmit script adds a submission-time validation check. The onCellEdit script separately governs State changes made from the list interface.

The resulting workflow is easy to trace: High Impact activates additional controls; missing Assigned To prevents submission; changing away from High reverses the conditional UI behavior; list-based State editing is blocked; and normal form-based State changes remain available.

The implementation should nevertheless be validated through actual execution evidence before being treated as fully verified. The source document establishes the intended configuration, testing scenarios, and expected results, but it does not document completed test evidence or broader enforcement outside the client-side scope.

10. Conclusion

The Implement Client Script & UI Policy (Incident) micro project demonstrates a practical method for improving Incident data quality through ServiceNow client-side configuration. The combination of UI Policy, UI Policy Actions, onChange, onSubmit, and onCellEdit Client Scripts provides dynamic field behavior, automated updates, validation before submission, and controlled State editing.

For High-impact Incidents, the implementation requires Assignment Group, automatically sets Urgency to High, and makes Urgency read-only. It also prevents submission when Assigned To is missing. When Impact is reduced from High, the UI Policy reverses its conditional restrictions. Direct State changes through list editing are blocked, while State changes through the Incident form remain permitted.

The project therefore provides a concise technical demonstration of how user-interface controls can support consistency and completeness in Incident Management. The immediate priority for moving from demonstration to operational use is to execute and document the defined test scenarios, verify behavior in the intended environment, and determine whether additional enforcement is required beyond the client-side interface.

11. Source Basis

This detailed brief is based on the supplied project document titled “Implement Client Script & UI Policy (Incident).” The source document contains the project abstract, problem statement, objectives, technologies, configuration values, Client Script descriptions, functional test cases, expected results, workflow, and conclusion. This report preserves those documented requirements and expands their presentation into an executive-oriented, detailed format without introducing unsupported project outcomes.